

A Survey of AI Techniques for LLM Optimization – DIA1 Option A

Group members: Tom Bernhard, Gabriel Beziou, Ali Abouhachim-Alami, Youssef Benaddi, Maxime Bendavid.

Introduction.....	2
Application of AI Techniques.....	2
First Academic paper	2
Second Academic paper.....	4
Third Academic paper.....	7
Fourth Academic paper.....	10
Fifth Academic paper.....	13
Sixth Academic paper	15
Comparative Analysis.....	17
Implementation of a Selected Technique.....	19
Specific problem	19
Implementation	20
Genetic Algorithm	21
Architecture of the Implemented Algorithm.....	21
Results.....	22
Analysis of Results: Genetic Algorithm vs. Random Search	23
Reinforcement Learning	23
Markov Decision Process	23
Reward	24
Implementation	24
First Algorithm.....	24
Second Algorithm: Reward Shaping (Efficiency-Aware)	25
Third Algorithm: Deep Q-Learning (DQN)	25
Fourth Algorithm: Parallelization	25
Empirical Evaluation	26
Conclusion	27

Introduction

Gemini 3 (launched in November) has approximately 21,5 trillion parameters and it is trained on a dataset encompassing a significant portion of the public internet. Knowing this, we can easily understand that today, optimizing hyperparameters, learning rates, or data loading strategies is crucial.

Indeed, poor optimization of hyperparameters, learning rates, or data loading strategies can lead to millions of dollars of wasted computing time and months of research delays. Optimization is a major factor in reducing costs and time. Hyperparameters and learning rates are also key components of performance convergence. Their effective optimization leads to better performance and greater generalization. Furthermore, optimizing these hyperparameters plays a significant role in reducing the energy consumption associated with training these massive models. Finally, reducing training costs helps democratize machine learning model research, making it accessible to a wider audience rather than just a few large organizations.

The key challenges in optimizing these factors are the following:

- **Large and expensive search space:** The search space for optimizing dozens of hyperparameters is vast and non-convex. Evaluating a single configuration requires lengthy and expensive training (objective function evaluation), making traditional search methods computationally prohibitive.
- **Parameter interdependence:** The optimal setting of a parameter, such as the learning rate, is highly dependent on other parameters, such as batch size or the optimizer used. These complex and nonlinear interactions make finding the global optimum difficult.
- **Delayed and noisy feedback:** The true performance of a configuration (the final validation loss) is only known after significant training time (delayed feedback), and intermediate metrics are noisy. This complexity necessitates the use of advanced, data-efficient, and AI-driven decision-making techniques.

Application of AI Techniques

First Academic paper

[Population Based Training of Neural Networks](#)

Training deep neural networks is very sensitive to hyperparameters (learning rate, regularization, loss weights, etc.), and these are usually chosen either by parallel search (grid/random search: many models in parallel, one training run each) or sequential optimization (Bayesian optimization: many runs one after another, each informed by the previous). Both options are expensive: parallel search wastes compute on bad configurations, while sequential methods take too long because each new trial requires a full training run. In reinforcement

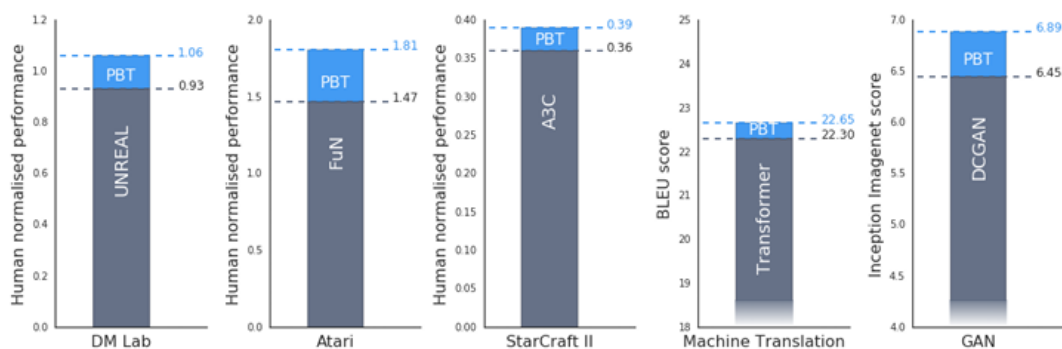
learning (RL) and other non-stationary settings, the best hyperparameters can also change during training, so searching for a single fixed set or a hand-designed schedule is often sub-optimal. The paper asks how to jointly optimize model weights and hyperparameters over time within a single training process, using a fixed compute budget, and without pre-specifying a hyperparameter schedule.

The authors propose **Population Based Training (PBT)**, an asynchronous, evolutionary-style method that combines ideas from parallel search and online adaptation. A population of models is created, where each member has its own weights θ_i and hyperparameters h_i (learning rate, dropout, loss weights). All population members are trained in parallel with standard gradient descent steps on the task at hand. Periodically, each “worker”:

- **Evaluates** its current performance using a metric of interest (validation score or episodic return).
- If it is underperforming, it **exploits** the population by copying the weights and hyperparameters of a better-performing model.
- It then **explores** by slightly perturbing or resampling the copied hyperparameters (for example multiplying the learning rate by 0.8 or 1.2) and continues training from these new settings.

This cycle of “train locally with SGD à exploit better models à explore new hyperparameters” runs throughout training. Because hyperparameters can change many times during a run, PBT effectively learns **hyperparameter schedules** (how they evolve over time) rather than a single fixed setting, and it automatically allocates more compute to promising models by copying their weights into weaker ones. The algorithm is fully asynchronous and needs only a shared store where workers read/write their current performance, weights, and hyperparameters.

The authors evaluate Population Based Training on several domains: deep reinforcement learning, machine translation, and GAN training.



In the figure above, each pair of bars compares a strong baseline without PBT (dashed line) to the same model trained with PBT (solid bar). The y-axis is always a task-specific performance metric: human-normalized score in DeepMind Lab and Atari, average episodic return in StarCraft II, BLEU score for machine translation, and Inception score for GANs.

In DeepMind Lab, PBT applied to the UNREAL agent improves the human-normalized performance of the best agent in the population from about 0.93 to 1.06. In **Atari**, Feudal Networks with PBT go from roughly 1.47 to 1.81. On **StarCraft II**, A3C with PBT also achieves a higher final score than the non-PBT version. For **machine translation** on WMT14 EnàDe with a Transformer, PBT slightly but consistently raises the BLEU score from 22.30 to 22.65, matching an expert-designed schedule without needing manual tuning. Finally, for **GANs** on CIFAR-10, PBT increases the Inception score of a DCGAN from 6.45 to 6.89, and the learned learning-rate schedules for generator and discriminator are quite different from standard hand-crafted ones.

Taken together, these results show that PBT reliably outperforms random search and fixed hyperparameter schedules across very different tasks. It not only improves final performance but also discovers effective, time-varying hyperparameter schedules automatically, providing a general way to make deep RL agents, sequence models, and GANs train faster and more robustly.

Second Academic paper

Learned Learning Rate Schedules for Deep Neural Networks

Training deep neural networks is very sensitive to the **learning rate (LR)**, which is the step size used by the optimizer when updating the model's weights. If the LR is too small, training becomes very slow; if it is too large, training can diverge. In practice, people often use learning-rate schedules (for example, starting with a larger LR and then decreasing it), but choosing the right schedule and its hyperparameters usually requires a lot of manual trial-and-error or expensive grid search. The paper addresses this question:

Can we automatically learn a good learning-rate schedule that makes training fast and stable, without manually tuning all the schedule hyperparameters?

In the author's approach, learning-rate scheduling is formulated as a **reinforcement learning (RL)** problem in which the system continuously decides how to update the learning rate to optimize the model's performance.

In RL, an agent interacts with an environment, observes its state, takes actions, and receives rewards that indicate how good those actions were. The goal is to learn a policy (a rule) that chooses actions to maximize the long-term reward.

Here, the environment is the training process of a neural network, and the agent acts as a learning-rate scheduler:

- At each **epoch** (one full pass through the training data), the agent receives an **observation** containing:
 - the current validation loss (how badly the model is performing on a held-out set),
 - how many epochs are left,

- and the gradient norms of each layer (a measure of how large the updates are)
- Based on this observation, the agent chooses an **action**: a single scalar value for the global learning rate to use in the next epoch.
- After training for that epoch with the chosen learning rate, the environment returns a **reward**.

This reward is higher when:

- The validation loss is lower (the model is learning well)
- The model reaches low loss earlier in training (fast convergence)

If training diverges (for example, the loss becomes NaN), the agent receives a large negative reward to strongly penalize that schedule.

To learn the policy that maps training statistics to learning rates, the authors use **Proximal Policy Optimization (PPO)**, which is a reinforcement-learning algorithm that trains a policy network using stable, small gradient-based updates. In this setting, the policy is a small neural network that, at every epoch, takes as input an observation describing the current training state, for example the validation loss, the number of epochs remaining, and the layer-wise gradient norms. It then outputs a single action: the learning rate to use for the next epoch. This also includes the very first learning rate at the beginning of training, so no initial LR must be selected manually.

During the PPO training phase, the policy is improved by rewarding schedules that produce fast and stable convergence, and by penalizing schedules that lead to divergence. Once PPO has finished training, the policy becomes a plug-and-play learning-rate scheduler. In a normal training loop, you simply feed the current statistics into this fixed policy at each epoch, and it returns the recommended learning rate. This removes the need for manual schedule design or large hyperparameter grid searches.

The authors evaluate their learned learning-rate scheduler on image-classification tasks (MNIST and CIFAR-100) and compare it to standard optimizers and manually designed schedulers. They keep the total “tuning budget” comparable: the RL agent is trained using roughly the same number of trainings runs that a grid search would use.

The key findings are:

1. **Stable training without divergence**

The RL-based scheduler almost never causes divergence, while many combinations of optimizers and manually tuned schedules do. This is particularly visible for aggressive schedules or when the initial LR is not well chosen.

2. **Competitive or better performance under the same budget**

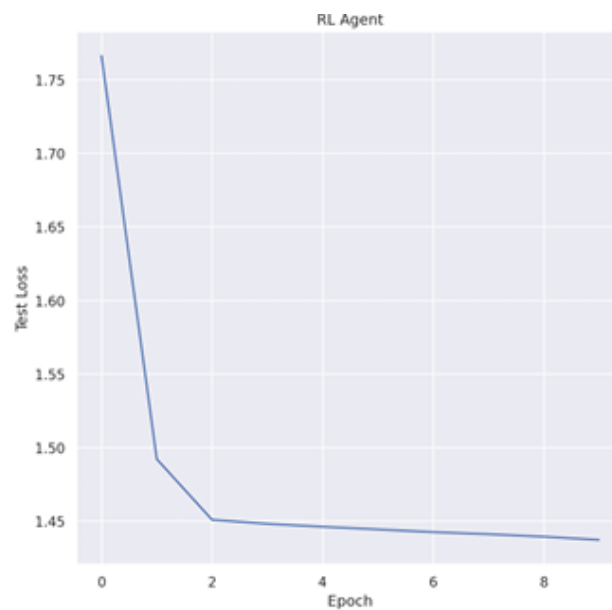
When comparing the final validation loss, the learned schedules are often as good as or even better than the best manually tuned scheduler found through grid search under the same

compute budget, and they clearly outperform typical off-the-shelf schedules when no extensive hyperparameter search is performed.

3. Non-trivial schedules

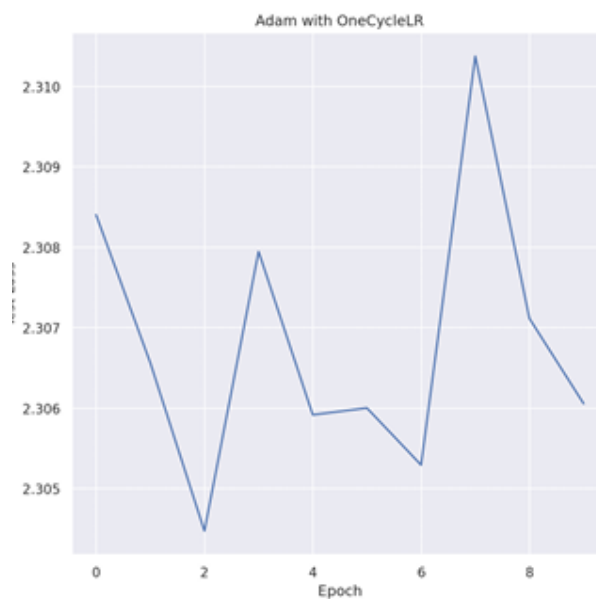
The learning-rate curves produced by the RL agent are not simple monotonic decays. They sometimes increase, then decrease, or change abruptly, in ways that would be hard to guess manually. Different training runs produce similar-shaped schedules, which suggests the agent has learned a consistent strategy rather than random behavior.

Validation losses on CIFAR-100 image classification dataset



(a) RL Agent

The validation loss decreases steadily from about 1.75 to 1.45 with no spikes or instability, showing the expected learning curve and demonstrating that the RL-based scheduler provides consistent, stable training across runs



(b) Adam + OneCycleLR

The loss oscillates sharply with a spike around epoch 6, indicating instability caused by a poorly matched schedule, showing that even Adam can perform badly without careful tuning.

Overall, the paper shows that a reinforcement-learning agent can serve as an automatic learning-rate scheduler: it learns a policy that chooses LR's epoch by epoch, leading to faster and more robust training than fixed hand-designed schedules, while reducing the amount of manual hyperparameter tuning needed. In 2025, this approach remains relevant, especially in settings where training stability is critical or where extensive hyperparameter search is too costly. While modern large-scale models often rely on sophisticated optimizers and well-tested LR schedules, a learned scheduler can still be valuable for smaller or mid-sized models, custom architectures, or research environments where training dynamics are unpredictable. It is particularly useful when compute budgets are limited, because it replaces expensive grid searches with a single learned policy that generalizes across runs.

Third Academic paper

BOHB: Robust and Efficient Hyperparameter Optimization at Scale

Modern deep learning models, including large language models (LLMs), require choosing many hyperparameters before training. Hyperparameters are settings that control how the model learns, such as the **learning rate** (the step size used in optimization), the **batch size** (how many samples are processed at once), or the **number of layers** in the network.

Selecting the right hyperparameters is called **hyperparameter optimization (HPO)**, and it has a major impact on both the final accuracy of the model and the efficiency of training.

There are two major families of methods traditionally used for HPO:

- **Bayesian Optimization (BO):** a model-based search technique that tries to predict which hyperparameter settings are promising. It guides the search intelligently, but each evaluation usually requires fully training the model, which is very expensive for deep networks.
- **Hyperband (HB):** a fast “multi-armed bandit-style” method that saves time by testing many hyperparameter configurations with very small training budgets (for example just a few epochs). Then it only continues training the best ones. This makes it fast, but Hyperband selects new configurations mostly at random and does not learn from previous trials.

Because of this, BO is smart but slow and expensive, while Hyperband is fast but not very informed. The key question the paper addresses is:

How can we combine the speed of Hyperband with the guided, model-based intelligence of Bayesian Optimization, in a scalable way that works with many hyperparameters and multiple parallel workers?

The authors introduce **BOHB (Bayesian Optimization with Hyperband)**, an algorithm that combines the strengths of two existing approaches to hyperparameter optimization. The idea is to keep Hyperband's speed while adding the guided search capabilities of Bayesian Optimization.

BOHB keeps Hyperband's use of **multi-fidelity evaluations**. A fidelity represents a training budget, such as a small number of epochs or a reduced dataset size. Hyperband uses a procedure called **Successive Halving**, where it evaluates many configurations on a small budget, selects the top performers, and then gradually increases the budget for only those survivors. This allows the algorithm to eliminate weak configurations early while keeping computation affordable.

To add intelligence to the search process, BOHB uses a Bayesian Optimization method called **Tree-structured Parzen Estimator (TPE)**. Instead of selecting new hyperparameter configurations at random, TPE models two probability distributions:

- one for hyperparameter values associated with good performance
- one for hyperparameter values associated with poor performance

New configurations are chosen from regions that the model considers promising, giving BOHB a guided search strategy rather than a purely random one.

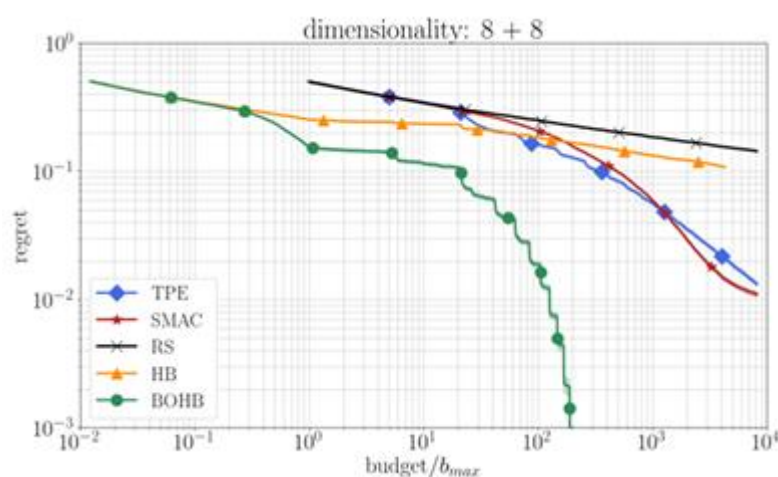
BOHB merges the two ideas by letting Hyperband decide the budgets and number of evaluations, while TPE proposes most of the new configurations. A small fraction of configurations is still sampled randomly to maintain exploration, but the majority comes from the model of past evaluations. BOHB also fits its model using data from the highest budget level with enough results, which improves the quality of its guidance. Overall, this design allows BOHB to be both fast and intelligent: it benefits from cheap early evaluations and also focuses its computational effort on the most promising regions of the hyperparameter space.

The authors evaluate BOHB on a wide range of benchmarks that cover very different types of machine-learning tasks. These include simple **toy functions** (synthetic test problems), classical **SVM classifiers on MNIST** (a digit-recognition dataset), **feed-forward and Bayesian neural networks, deep reinforcement-learning tasks trained with PPO**, and **convolutional neural networks (CNNs) on CIFAR-10**, which is a standard image-classification dataset. Testing BOHB on such diverse cases shows how well the method generalizes across models and domains.

The experiments demonstrate several important results. First, BOHB **matches or exceeds the anytime performance of Hyperband**, which means that it finds good hyperparameter configurations very quickly when the available training time is limited. "Anytime performance" refers to how good the results are at different moments during the search, not only at the end. Second, BOHB **converges to better final configurations** than Hyperband or random search, and it also often outperforms traditional Bayesian Optimization methods, especially when each evaluation (i.e., each model training) is expensive. This shows that BOHB not only starts strong but also finishes strong.

Finally, BOHB **scales effectively** when the number of hyperparameters increases or when many parallel workers are used. In practice, this means that BOHB can evaluate many configurations at once and achieves almost linear speedups when more compute resources are added, making it suitable for large-scale machine-learning settings.

These trends are clearly illustrated in this figure



This compares BOHB with several hyperparameter optimization methods on a challenging 16-dimensional search space (8 categorical and 8 continuous hyperparameters). The x-axis represents the available training budget, expressed as a fraction of the maximum allowed budget, and the y-axis shows regret, which measures how far the current best configuration is from the optimal one. Lower regret indicates better performance.

The results show a clear advantage for BOHB. At very small budgets, BOHB already outperforms Random Search, Hyperband, and the model-based methods (TPE and SMAC), illustrating its strong **anytime performance**. As the budget increases, BOHB improves rapidly and its regret curve drops significantly faster than the competing methods. At high budgets, BOHB achieves the lowest regret overall and is the only method to approach near-zero regret, demonstrating both robust convergence and superior final performance. This figure highlights BOHB's ability to combine Hyperband's fast low-budget evaluations with the guided search of Bayesian Optimization, resulting in consistently better hyperparameter configurations throughout the entire optimization process.

Overall, the paper shows that BOHB is a practical and powerful hyperparameter optimization method that successfully combines the strengths of Bayesian Optimization and Hyperband. It achieves fast initial progress by using cheap, low-budget evaluations, and it uses its learned probabilistic model to focus the search on promising regions of the hyperparameter space, leading to strong final performance while remaining efficient and scalable.

Fourth Academic paper

Learning to Teach

The lifecycle of a large language model (LLM) is computationally expensive, often involving billions of tokens and thousands of GPU hours. The data loading strategy is a critical inefficiency in this process. Traditional training pipelines treat each data point as having equal value, feeding the model randomly or in turn. However, this method ignores a fundamental reality of learning: not all examples are equally instructive. Some data is redundant, some are too noisy, and some are too difficult for a novice model.

This study examines how AI decision-making techniques are applied to optimize this process. Specifically, it analyzes how intelligent agents can dynamically organize a model's "schedule", effectively "teaching" it rather than simply letting it learn from raw data.

The paper proposes moving from heuristic-based learning to a fully learned, agent-based approach. First, the central problem identified is the rigidity of traditional machine learning optimization. In fact, in standard training such as stochastic gradient descent (SGD), the program is static, which means that the data loader blindly collects batches without considering the current state of the model. This leads then to two forms of inefficiency: Redundancy, as the model wastes computational resources by training on samples it already masters, and interference because the model may be exposed too early to complex or noisy samples, which can destabilize gradients and slow convergence.

The authors argue that a human teacher dynamically adjusts their teaching materials based on student feedback (like test scores). In contrast, standard AI training lacks this feedback loop between the data (the material) and the optimizer (the teacher).

The idea that comes to mind after this observation is the use of Deep Reinforcement Learning through a framework named "Learning to Teach" (L2T). They view the learning process as a sequential decision-making process. The mechanism of how it works is as follows:

Instead of a random sampler, a teacher agent (a separate neural network) lies between the dataset and the student model. For each batch of data, the teacher decides which specific samples should be passed to the student for learning and which should be rejected.

We can formulate this method as a classical reinforcement learning algorithm like seen in our course, transforming the training loop into a standard RL tuple (S, A, R, π) with:

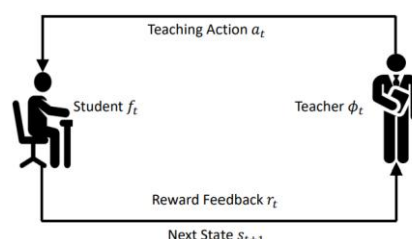


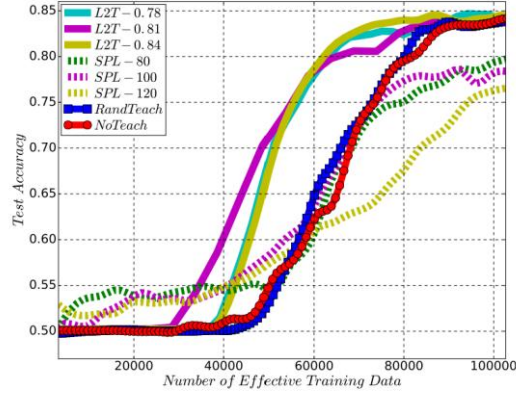
Figure 1: The interactive process between teacher and learner.

- The environment: Here, this is the student model combined with the incoming data stream. It is dynamic because the student's weights change at each step.
- The state (S_t): The teacher observes a complex representation of the state containing student characteristics like current iteration number and validation accuracy history, and data characteristics like the "difficulty" of the data point (for example, loss value, margin between the correct class and the nearest rival).
- The action (A_t): A discrete binary choice for each data point: Keep (1) or Delete (0).
- The reward (R_t): Unlike instant-point games seen in the TD sessions, the reward here is rare and deferred. The teacher receives a positive signal if the student achieves a target accuracy τ in fewer steps than a reference.
- The policy (π_θ): The teacher's policy follows a Bernoulli distribution. Mathematically, $\pi(a|s) = \text{Bernoulli}(\sigma(W \cdot s + b))$. The output is a probability $p \in [0,1]$ derived from a sigmoid function, determining whether to keep the data point (the decision space is discrete)
- Optimization: The teacher model uses reinforce (policy gradient) to adjust its parameters. It learns to maximize the probability of selecting data sequences that allow for the fastest student convergence.

While solved by reinforcement learning, the problem definition is structurally a dynamic constraint satisfaction problem (CSP). Indeed, CSP provides the structure (what constitutes a valid and efficient selection):

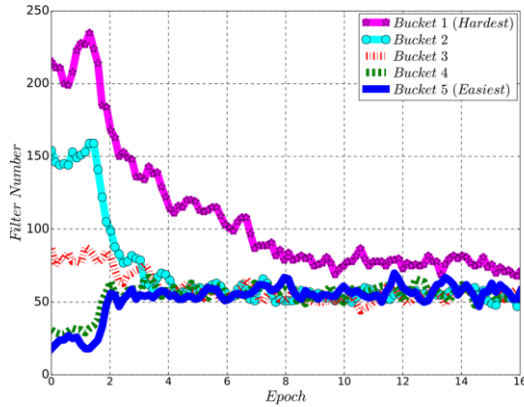
- Variables (X): The set of potential training samples in a mini batch $\{x_1, \dots, x_n\}$.
- Domains (D): The domain for each variable is binary $\{0, 1\}$ (Include/Exclude).
- Constraints (ϕ):
 - Hard Constraint (Budget): The student has a finite computational budget (time/steps). The teacher cannot simply feed all data; it must select a subset.
 - Soft Constraint (Objective): The selected subset must satisfy the condition of maximizing the gradient direction towards the global minimum.
- The Solver: Instead of using backtracking or arc consistency (which are too slow for real-time training), the RL Agent acts as a heuristic solver. It approximates the optimal assignment of $\{0, 1\}$ values to the batch variables to satisfy the objective function (learning speed) under the constraint of limited steps.

According to the document, the L2T method offers four specific improvements to the training process. Each improvement is supported by precise experimental results presented in the article. The first one is data efficiency. The method significantly reduces the amount of data required to reach a target accuracy. By filtering out easy or redundant examples, the model focuses only on the most informative data points. Evidence of this conclusion is for example in the figure 2 (c) of the paper where L2T reduces the required data to 75% of the baseline, demonstrating that intelligent data selection works for sequence data (crucial for LLM).



(c) IMDB

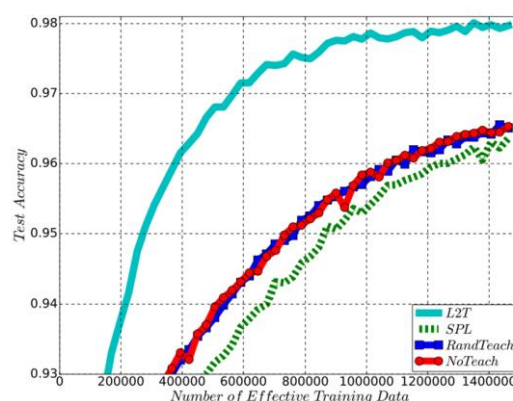
The second improvement is that the system does not rely on fixed heuristics (such as "always choose difficult data"). On the contrary, the reinforcement learning agent learns an optimal strategy specific to the type of data. It is shown in figure 3 (c) where the teacher spontaneously adopts a curriculum learning strategy, starting with easy examples and gradually introducing harder ones.



(c) IMDB

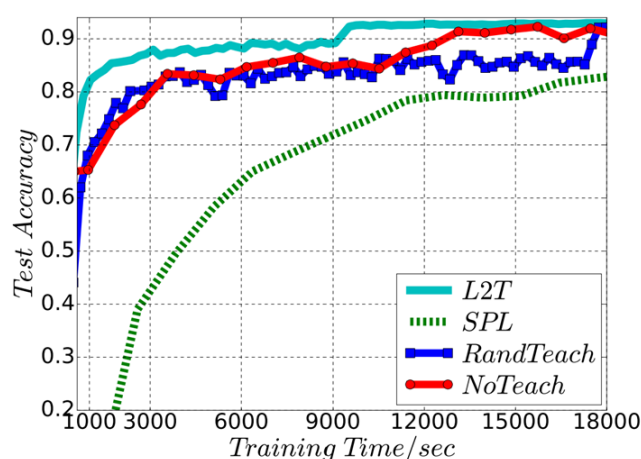
Note: This is also relevant and proves the RL agent is not just a static filter; it dynamically adjusts the constraint satisfaction criteria based on the learning phase of the student.

In third, L2T improves training scalability by allowing a teacher agent, trained on a small, inexpensive model, to teach a larger, more expensive model. This is illustrated in figure 4 (c) because a teacher trained on CIFAR-10 (Images) successfully optimizes the training of an MLP on MNIST. This suggests the Teacher learned a universal meta-policy about loss management rather than just memorizing pixel patterns.



(c) CIFAR10→MNIST

Finally, it reduces the actual training time and helps the model escape local minima to achieve greater accuracy. In figure 5, the learning time (L2T, cyan curve) progresses faster than the reference curve (red curve).



Fifth Academic paper

Hyperparameter Optimization for Large Language Model Instruction-Tuning

Large Language Models (LLMs) like LLaMA-2 can be adapted to follow user instructions through instruction-tuning, where the model is fine-tuned on pairs of instructions and desired outputs. Because these models have billions of parameters, fully fine-tuning them is extremely expensive. A common solution is **parameter-efficient fine-tuning (PEFT)**, where most weights are frozen and only a small set of extra parameters is trained. This paper focuses on **LoRA (Low-Rank Adaptation)**, a PEFT method that adds small low-rank matrices on top of the frozen weights. However, LoRA's performance is very sensitive to several hyperparameters (for example its rank, a scaling factor α , dropout, and the learning rate), and picking them by hand is tedious and often sub-optimal. The authors ask how **automatic hyperparameter optimization** can improve instruction-tuning of an LLM (LLaMA-2-7B) while staying within a limited compute budget.

The authors treat instruction-tuning with LoRA as a hyperparameter optimization problem. In practice, this means:

1. You choose some LoRA hyperparameters (rank, scaling α , dropout, learning rate)
2. You fine-tune LLaMA-2 with these values for a small number of epochs
3. You measure how good the model is, mainly with the validation loss
4. An optimizer uses this result to propose a new better set of hyperparameters
5. You repeat this loop until you run out of compute budget

The whole fine-tuning process is seen as a black box: we do not try to model the internals of the network, we only care about the input (hyperparameters) and the output (validation loss).

To drive this black-box search, the paper compares **two optimization algorithms** that represent two different philosophies:

1. **NOMAD (MADS) – direct search**

NOMAD is a **direct search method**, which you can imagine as a clever trial-and-error strategy in the hyperparameter space.

- It keeps track of the current best hyperparameter setting
 - Around this point, it tries small variations (for example, slightly higher rank, slightly lower learning rate) and checks whether the validation loss improves
 - If it finds a better point, it moves the “current best” there and repeats the process with finer adjustments; if not, it explores a bit more widely
- This gradual refinement on a “mesh” of candidate points is what the **Mesh Adaptive Direct Search (MADS)** framework provides. It does not build a probabilistic model of the objective; instead, it relies purely on function evaluations (fine-tuning runs) and a systematic exploration pattern

2. **NNI-TPE – model-based Bayesian optimization**

NNI-TPE (Tree-structured Parzen Estimator in Microsoft’s NNI toolkit) represents the **Bayesian Optimization** approach. Instead of only doing trial and error, it **learns a probabilistic model** of which hyperparameter regions look promising

- After several runs, it separates past configurations into “good” ones and “bad” ones, based on their validation loss
- It then builds two probability distributions: one describing where good results tend to lie, and one describing where bad results tend to lie
- New hyperparameters are chosen in places where the “good” distribution is high and the “bad” one is low, which means they are likely to improve performance

In both cases, the outer loop is the same: propose hyperparameters → fine-tune the LLM with LoRA → measure validation loss → update the optimizer. The difference is how they decide what to try next: NOMAD uses structured trial-and-error about good points, while TPE uses a learned probability model of good vs. bad regions.

The authors first tune LoRA hyperparameters using validation loss on an instruction-tuning mixture built from **Alpaca** and **Dolly** datasets, which are collections of instruction–response pairs designed to teach an LLM to follow natural-language instructions. For this tuning stage, both NOMAD and NNI-TPE find configurations that clearly beat the default LoRA settings; NNI-TPE usually discovers the single best validation loss, often with surprisingly low LoRA ranks, while NOMAD tends to concentrate on slightly higher ranks but still identifies strong candidates.

To see whether these gains transfer beyond validation loss, the paper evaluates the tuned models with **InstructEval**, a suite of downstream benchmarks. These include **MMLU** (a multi-task exam-style benchmark covering many academic subjects), **BBH** (“Big-Bench Hard”, a set of challenging reasoning tasks), **DROP** (a reading-comprehension dataset requiring numerical reasoning over text), and **HumanEval** (a code-generation benchmark where models must write small Python functions). On these benchmarks, models tuned by NOMAD or NNI-TPE generally outperform both the untuned base model and the model trained with default LoRA hyperparameters, although the configuration with the best validation loss is not always the best on every benchmark.

Finally, the authors run a small human evaluation using a subset of the **Vicuna preference dataset**, where humans compare responses from two models and pick the better one. In this setting, a NOMAD-tuned model is preferred over the default-tuned model in about 55% of cases, indicating a noticeable improvement in perceived output quality and alignment. Overall, these results show that black-box hyperparameter optimization around LoRA can meaningfully improve both automatic scores and human preferences for instruction-tuned LLMs, compared to using hand-chosen default hyperparameters.

Sixth Academic paper

[Efficient Hyper-Parameter Search for LoRA – \(BO + LLM for LoRA HPO\)](#)

Fine-tuning Large Language Models (LLMs) with **LoRA (Low-Rank Adaptation)** is much cheaper than full fine-tuning, because only small low-rank adapter matrices are trained while the original weights are frozen. However, LoRA is highly sensitive to hyperparameters such as the rank r , scaling factor α , learning rate, batch size, and dropout rate. Performance can change a lot depending on how these are combined, and the search space easily explodes to tens of thousands of configurations. Exhaustive or naive search is therefore too expensive. Classical **Bayesian Optimization (BO)** is attractive for this kind of expensive black-box optimization, but it usually assumes a continuous input space and does not exploit domain knowledge about LoRA, while each evaluation (a full LoRA fine-tuning run) is still costly. The paper asks how

to design a more efficient hyperparameter search for LoRA that uses domain knowledge, works with discrete and structured hyperparameters, and reduces the cost of each evaluation.

The authors propose a language-aided Bayesian Optimization framework for tuning LoRA hyperparameters. The key idea is instead of running Bayesian Optimization directly on raw hyperparameter values (rank, α , learning rate, etc.), they first turn each configuration into a **text description**, let an LLM encode this description into a **vector representation**, and then run BO in that vector space.

Concretely, each LoRA configuration is written as a **structured text template** that lists the hyperparameters and includes short natural-language explanations of what they do and how they interact (for example, that the scaling α behaves like a gain on the updates, or that rank and α are often chosen together). This text is fed into a LLM, which produces an embedding, which is a numerical vector that captures the meaning of the text. This is what we call an **LLM-based embedding**: a dense vector representation produced by the LLM from the hyperparameter description.

To make this embedding more expressive for LoRA tuning, the input text also includes a **learnable token**. A token is normally just a unit of text, a learnable token here means a special placeholder whose vector is not fixed but trained along with the BO framework. Over time, this token can capture LoRA-specific knowledge that the plain LLM embeddings do not encode by default. In addition, the authors add a small **projection layer**, which is simply a shallow neural network (often a linear layer) that takes the LLM embedding (plus the learnable token) and maps it into a new vector space that is better suited for modeling the objective. You can think of it as a trainable adapter that shapes the representation specifically for hyperparameter optimization.

Bayesian Optimization then works on these projected embeddings instead of directly on the raw hyperparameters. In practice, the method learns a **Gaussian Process**, which is a statistical model used to approximate an unknown function. Here, that unknown function is “embedding $\rightarrow$ performance” (for example, validation loss). For any new embedding, the Gaussian Process provides both a predicted performance and a measure of uncertainty about that prediction. You can think of it as a smooth curve fitted through all the trials seen so far, with error bars that say how confident the model is in each region of the space.

Using this model, an **acquisition function** chooses which point in the embedding space is the most promising to test next, for example a point that is predicted to have low loss or one where the uncertainty is high and more information would be useful. Once Bayesian Optimization has selected such a point in the embedding space, the system goes back through the same text plus LLM pipeline to recover the corresponding concrete hyperparameter configuration, and this configuration is then evaluated by running a new LoRA fine-tuning run.

Finally, to reduce the cost of each evaluation, the authors use a **proxy training evaluation**. Instead of fine-tuning on the full dataset (which is expensive), they fine-tune on a smaller random subset (for example 10% of the data) and use the validation performance on this subset as a proxy for full-data performance. A proxy here means an approximate objective that is

cheaper to compute but still strongly correlated with the true objective. This allows the BO loop to run many more trials under the same compute budget, while still being guided toward hyperparameters that work well when later trained on the full dataset.

The authors evaluate their language-aided Bayesian Optimization framework on several LoRA-style adapters (PiSSA and variants) and multiple LLM backbones, including LLaMA2-7B, Mistral-7B-v0.1, and Gemma-7B. They compare against the **original hyperparameter settings** proposed in the PiSSA paper and measure performance on four benchmarks: **GSM8K** and **MATH** for mathematical reasoning, and **HumanEval** and **MBPP** for code generation (Pass@1 means the fraction of problems solved correctly on the first try).

Model	Ours	Accuracy (%)		Pass@1	
		GSM8K	MATH	HumanEval	MBPP
LLaMA2-7B (Touvron et al. 2023)	✗	41.47	5.24	16.31	35.47
	✓	62.93 (+21.46)	12.88 (+7.64)	30.49 (+14.18)	42.59 (+7.12)
Mistral-7B-v0.1 (Jiang et al. 2023)	✗	69.90	19.96	45.73	61.90
	✓	74.07 (+4.17)	23.46 (+3.5)	54.27 (+8.54)	65.08 (+3.18)
Gemma-7B (Team et al. 2024)	✗	75.51	29.44	49.39	63.23
	✓	78.77 (+3.26)	30.24 (+0.8)	53.05 (+3.66)	67.46 (+4.23)

As summarized in Table above, their tuned hyperparameters consistently outperform the original ones across all models and tasks. For example, on LLaMA2-7B, GSM8K accuracy increases from 41.47% to 62.93% (+21.46 points) and HumanEval Pass@1 improves from 16.31% to 30.49% (+14.18). Similar gains are observed for Mistral-7B and Gemma-7B, with improvements of several percentage points on both math and coding benchmarks. Overall, these results show that the proposed LLM-aided Bayesian Optimization method finds substantially better LoRA hyperparameters than hand-designed settings, and that the benefits transfer across different backbone models and evaluation tasks.

Comparative Analysis

The table below summarizes the six selected methods along a common set of criteria. The first four papers (PBT, Learned LR Schedules, BOHB and Learning to Teach) are closely aligned with the core algorithms studied in the course: search, local search and genetic algorithms, and reinforcement learning. The last two papers (LoRA HPO and LLM-aided LoRA BO) build on these ideas but apply them in more advanced, LLM-specific settings, showing how classical AI optimization techniques are now used to tune modern large language models.

Comparative overview of the six selected methods

Paper name	Problem focus	Key idea	Link to the course
Population based Training	Dynamic training hyperparameters	Population of models that copy and mutate hyperparameters from the best ones.	Genetic Algorithms / Local Search
Learned LR Schedules	Learning-rate schedules	RL agent chooses the learning rate at each	Reinforcement Learning / MDPs

		epoch from training statistics.	
BOHB	Hyperparameter search under limited budget	Combine many cheap partial trainings with a guided search over configurations.	Search & Exploration-Exploitation
Learning to Teach	Data / curriculum strategy	Teacher model learns which examples or tasks to show to the student.	Reinforcement Learning / Sequential Decisions
LoRA HPO (NOMAD + TPE)	LoRA hyperparameters for LLM fine-tuning	Treat LoRA instruction-tuning as a black box and search its hyperparameters.	Black box / Heuristic Search
LLM-aided LoRA BO	Efficient LoRA hyperparameter search	Use LLM-based embeddings of hyperparameters inside Bayesian Optimization.	Bayesian-style Search & Representation Learning

The table highlights the distinct trade-offs between these approaches. PBT and the PPO-based scheduler focus on online adaptation by adjusting hyperparameters in real-time. While this improves stability and final performance, it demands significantly more engineering effort and computing. Contrarily, BOHB and the LoRA HPO methods function as offline black-box optimizers. They rely on multiple short runs to identify optimal static parameters, making them easier to integrate into existing pipelines even if they lack the ability to react to non-stationary dynamics. Learning to Teach takes a different angle by optimizing the data curriculum rather than the optimizer itself, accepting the complexity of a teacher model in exchange for better data efficiency. Ultimately, while evolutionary and RL methods offer fine-grained control at a higher cost, Bayesian-style approaches provide a more automated path to performance gains.

A second way to compare the papers is to look at **generality versus specialization**. PBT and BOHB are quite general. In principle they can be used with many types of models and tasks, for example reinforcement learning or standard supervised learning. The PPO-based learning rate scheduler is already a bit more specific, because the agent is trained for one particular optimizer and training setup. The two LoRA HPO papers are clearly specialized: they are built for instruction-tuning large language models with adapters, and this is exactly where they give the biggest gains. Learning to Teach is somewhere in the middle. The teacher–student idea is quite general, but in practice the experiments are tailored to particular image or text tasks. In short, PBT and BOHB are good candidates when we design a new system from scratch, while the LoRA methods make more sense when the architecture is fixed and the main problem is the cost of fine-tuning a large language model.

If we look at **cost and practicality**, the picture also changes. PBT usually assumes a fairly large computing budget with several workers running in parallel. It works best when we can afford to train many models at the same time. The PPO-based learning rate scheduler is expensive to train once, but the learned policy is then cheap to reuse across later runs. BOHB

is designed for the situation where each training run is expensive and the total budget is limited, and it fits well into existing code that can launch multiple short trainings. The LoRA HPO methods try to save even more computing by training on smaller subsets of the data and, in the LLM-aided BO paper, by using a better representation of hyperparameter configurations. The price to pay is a more complex pipeline that depends on a strong pretrained language model.

From our point of view, each family of methods makes sense in different scenarios. With a large reinforcement learning system or any training process that changes a lot over time. We would prefer PBT or the PPO-based scheduler, because adapting hyperparameters on the fly is more helpful than finding one static “best” setting. For a more standard supervised model with a fixed number of runs, BOHB is a good default, since it balances performance and implementation effort. When the goal is to fine-tune a large language model with LoRA and compute is the main bottleneck, the two LoRA HPO approaches are the most attractive, because they are designed specifically for this situation and consistently beat hand-tuned baselines. Learning to Teach is most interesting when data itself is expensive, for example when labels are costly and we want to learn as much as possible from fewer training examples.

These comparisons also suggest a few **future research directions**. One natural idea is to combine online and offline methods, for instance by using BOHB or the LLM-aided BO approach to find good starting hyperparameters, then refining them during training with PBT or a learning-rate scheduler based on reinforcement learning. Another idea is to reuse the language-model-based representation from the LoRA HPO papers for other optimization targets, such as learning-rate schedules or data curricula, so that language models help design more parts of the training process. Finally, it would be useful to study more systematically how much we can transfer hyperparameters across models, for example from a smaller model to a larger one, in order to further reduce the cost of hyperparameter search for future large language models.

Implementation of a Selected Technique

Specific problem

We chose to implement automated joint Hyperparameter Optimization (HPO) and Neural Architecture Search (NAS) for efficient LLM pre-training.

The training of Large Language Models (LLMs) is notoriously sensitive to the choice of hyperparameters and architectural configurations. A suboptimal selection can lead to training instability (divergence), slow convergence, or poor generalization, resulting in a waste of significant computational resources (GPU hours) and energy.

This sub-problem specifically addresses the challenge of navigating a high-dimensional, non-convex search space to simultaneously identify:

- **Architectural Constraints:** The optimal model depth n_{layer} and width n_{embd} that balance expressivity with computational loss.

- Training dynamics: The ideal learning rate *lr* and batch size *batch_size* to ensure stable and rapid convergence.
- Regularization: the appropriate dropout rate *dropout* to prevent overfitting on the training data.

Why this is a critical optimization problem:

- Interdependence: These parameters are not independent (for example, a deeper network often requires a different learning rate or dropout intensity). Their isolated (sequential) optimization often yields suboptimal results compared to joint optimization.
- Cost of evaluation: Unlike classical optimization problems where evaluating the objective function is inexpensive, evaluating a single “solution” in this context requires training a neural network, a computationally intensive operation. Therefore, “sampling efficiency” (finding the best configuration with the fewest possible training iterations) is the main success criterion for any AI technique applied here.

Implementation

Since our team is made of 5 students and not 4, we decided to implement not one but two different AI techniques to tackle this problem.

Since we can't train an entire LLM, we are going to use NanoGPT which is one of the simplest and quickest implementations to train or fine-tune a GPT model.

Note: GPT means Generative Pre-trained Transformer.

LLMs are trained for a very specific task: Next Token Prediction. It means that LLMs like NanoGPT are trained to read a chunk of text and guess the very next character.

For example:

- Input: “To be or not to b”
- Target: “e”

Our objective is to minimize the loss function. Indeed, we measure the performance of a model using Cross-Entropy Loss, which quantifies how “surprised” the model is by the actual next character. Here we are going to train NanoGPT on the TinyShakespeare text data and try to lower the Loss as much as we can.

To lower this loss, we must tune the "recipe" of the model. This is what our Agents are going to learn to do:

- Learning Rate (*lr*): Controls how fast the model updates its brain. Too high = chaos; too low = too slow.
- Batch Size: How many examples it sees at once.
- Layers (*n_layer*) & Embedding (*n_embd*): The size and complexity of the model.

To do that we chose to implement a Genetic Algorithm (GA) and a Reinforcement Learning (RL) algorithm. They have different search strategies and with this experiment we are going to be able to do a comparative analysis of both methods.

Genetic Algorithm

To address the complex optimization landscape of LLM hyperparameter tuning, we selected the **Genetic Algorithm (GA)**, a bio-inspired heuristic search method discussed in Lecture 5. While gradient-based methods are standard for training model weights, they are unsuitable for hyperparameter optimization because the loss landscape with respect to parameters like *n_layer* or *batch_size* is non-differentiable and highly multimodal.

The GA treats the problem as an evolutionary process where a population of candidate configurations evolves over generations. This approach offers distinct advantages for our "Proxy Model" scenario:

1. **Global Search Capability:** Unlike Hill Climbing (Lecture 4), which can easily get trapped in local optima, the GA maintains a diverse population, allowing it to explore multiple regions of the search space simultaneously.
2. **Parallelism:** The evaluation of individuals is independent, making it scalable (though we implemented a sequential version for this proof-of-concept).
3. **Flexibility:** It handles mixed search spaces (continuous variables like *learning_rate* and discrete variables like *batch_size*) seamlessly without requiring gradient information.

Architecture of the Implemented Algorithm

Our implementation follows the standard evolutionary cycle defined in the course material: **Initialization** → **Evaluation** → **Selection** → **Reproduction (Crossover & Mutation)**. We tailored each step to the specific constraints of training the NanoGPT model.

Step 1: Genome Representation & Initialization

We defined a mixed search space where each individual (candidate solution) is represented by a "genome" (a dictionary) containing five genes:

- **Continuous Genes:** *learning_rate* (log-scale range [1e-4, 1e-2]), *dropout* ([0.0, 0.5]).
- **Discrete Genes:** *batch_size*, *n_embd*, *n_layer* (chosen from predefined sets like {16, 32, 64}).

The initial population of size N=20 is generated randomly to ensure maximum coverage of the search space.

Step 2: Fitness Function (The "Black Box")

The fitness function evaluates the phenotype (the actual performance) of a genome.

$$Fitness(x) = \frac{1}{1 + ValLoss(x)}$$

Note: Since the GA maximizes fitness, but we want to minimize the Loss, we use an inverse relationship.

For each evaluation, the algorithm instantiates the NanoGPT model with specific genes, trains it for a fixed budget (e.g., 200 iterations) on the dataset and returns the final validation loss.

Step 3: Selection Strategy (Tournament Selection)

To select parents for the next generation, we implemented **Tournament Selection** (as seen in Lecture 5).

- **Mechanism:** For each parent slot, we randomly select $k=3$ individuals from the population and choose the one with the best fitness.
- **Justification:** This method provides a tunable selection pressure. It avoids the scaling problems of Roulette Wheel selection (where a "super-individual" dominates too early) and is more robust than Rank selection.

Step 4: Reproduction (Elitism, Crossover & Mutation)

We designed a reproduction pipeline to balance **Exploration** (finding new areas) and **Exploitation** (refining good areas):

1. **Elitism (Exploitation):** The top 2 individuals of every generation are copied directly to the next generation without modification. This guarantees that the "Best-So-Far" solution is never lost (monotonic improvement).
2. **Uniform Crossover:** Two parents swap genes with a 50% probability per gene. This allows combining a good learning rate from Parent A with a good architecture from Parent B.
3. **Gaussian Mutation (Exploration):**
 - a. For continuous parameters ($lr, dropout$), we apply a multiplicative Gaussian noise: $x_{new} = x_{old} \cdot N(1, 0.2)$. This allows fine tuning.
 - b. For discrete parameters, we randomly sample a value from the allowed set with probability $p=0.25$. This prevents the population from stagnating.

Results

To rigorously evaluate the performance of our Genetic Algorithm, we benchmarked it against a **Random Search** baseline under identical constraints ("Ceteris Paribus").

- **Budget:** Both algorithms were allowed exactly **200 evaluations** (20 individuals times 10 generations).
- **Task:** Minimize Validation Loss on TinyShakespeare.

- **Metric:** We track the "Best Loss So Far" to measure convergence speed and final solution quality.

Analysis of Results: Genetic Algorithm vs. Random Search

Observation 1: Convergence Superiority

As shown in Figure A below, the Genetic Algorithm (Blue curve) demonstrates a clear "staircase" descent, consistently improving its best solution over time. Starting from a similar initial loss (approx. 2.53), GA successfully optimized the hyperparameters to reach a final loss of 2.519.

In contrast, the Random Search (Red curve) quickly hit a plateau around 2.522. While Random Search is effective in the early stages (finding a "good enough" solution rapidly), it lacks the mechanism to refine that solution. GA leverages the information from previous generations to "zoom in" on the optimal configuration.

Observation 2: Diversity and Exploration

We analyzed the population dynamics by plotting the Average Loss of the population (Green dotted line) alongside the Best Loss.

- The gap between the **Best** and the **Average** indicates that our high mutation rate (0.25) successfully maintained genetic diversity.
- The average loss did not collapse onto the best loss, meaning the population did not suffer from **premature convergence**. The algorithm continued to explore suboptimal regions (high average loss) while securing the best solution via elitism. This behavior validates the theoretical strength of GAs: escaping local optima through stochastic exploration.

Reinforcement Learning

We selected Reinforcement Learning (RL) because HPO can be effectively modeled as a Sequential Decision-Making Process. Unlike Grid Search, which treats parameters as independent isolated choices, RL recognizes that hyperparameters are deeply interdependent (e.g., the optimal Learning Rate depends heavily on the chosen Batch Size). By formulating the problem as an MDP where the agent sequentially selects parameters, the agent learns to identify advantageous combinations rather than just individual values. Furthermore, RL is particularly suitable for this "Black Box" optimization because it does not require gradients from the loss function; it learns purely through trial-and-error (reward feedback), allowing it to optimize non-differentiable metrics like validation loss or even discrete architectural choices (like n_layer) where standard gradient descent fails.

For the RL, we implemented 4 different algorithms. To begin, we drew inspiration from the Lab Session dedicated to RL.

Markov Decision Process

The first step was to define the Markov Decision Process (MDP):

- State Space (S): Represents the current decision step (choosing LR, then Batch Size, etc.).
- Action Space (A): Discrete choice for each parameter

Reward

The agent only gets a reward after it selects all parameters and the model trains. Our goal is to minimize the Loss, and the RL Objective is to maximize its Reward so:

$$R = \frac{1}{\text{Validation Loss}}$$

Implementation

First Algorithm

Step 1: Create the RL Environment (NanoGPTEnv).

We need to wrap the NanoGPT training code into a class that looks like the Gymnasium environments we used in Lab 5 (FrozenLake). So:

- `__init__`: Define our search space (the list of possible values for each hyperparameters)
- `reset()`: Reset the current state to 0 (start of choosing parameters).
- `step(action)`:
 - Record the chosen value for the current hyperparameter.
 - Move to the next state (e.g., from choosing LR to choosing Batch Size).
 - If Terminal State (all params chosen):
 - Trigger the NanoGPT training (using the selected params).
 - Get the *val_loss*
 - Compute Reward
 - Return *observation, reward, terminated = True, info*.

Step 2: Implement the Q-Learning Algorithm

Here is the logic, following what we saw in the Lab Session:

1. Initialize Q-Table: A table of size [Num_States, Max_Num_Actions]
2. Epsilon-Greedy Policy:
 - a. With probability of ϵ : Pick a random hyperparameter value (Exploration).
 - b. With probability of $1 - \epsilon$: Pick the value with the Highest Q-score (Exploitation).
3. Update the Rule: We use the Bellman Equation from lecture 7 (RL) to update the Q-values based on the reward received at the end of the training episode.

$$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max(Q(S', A) - Q(S, A))]$$

Step 3: The training loop (The “Main” function)

- Loop for N episodes.
 - Reset environment.

- Selection Loop: Agent picks LR → picks Batch Size → ... → picks Dropout.
- Train & Evaluate: The environment trains the NanoGPT and returns the loss
- Backpropagation Reward: update the Q-values for the choices made in this episode
- Decay Epsilon: Reduce exploration over time (start exploring, end exploiting)

The best Validation Loss for this Algorithm is around 2.52.

Second Algorithm: Reward Shaping (Efficiency-Aware)

In this Algorithm, the only change is the Reward. Instead of only optimizing for accuracy, we implemented Reward Shaping to find a balance between performance and computational cost.

$$R = \frac{10}{\text{Validation Loss}} - \text{Compute Cost} \cdot \text{Penalty}$$

Accuracy Reward: Inverse of the Validation Loss (lower loss = higher reward).

Unfortunately, this upgrade didn't work as well as we thought, indeed it took a bit less time ≈5min to execute but for a less good Validation Loss ≈ 2.57.

Third Algorithm: Deep Q-Learning (DQN)

In this Algorithm we will replace the Q-Table with a Neural Network.

- Input: The current state (e.g., "We are currently choosing the Learning Rate").
- Output: The predicted value of every possible choice.

This can benefit us because the Neural Network can learn patterns and relationships between parameters, potentially finding the optimal configuration faster.

Unfortunately, our implementation didn't really work. Indeed, our Algorithm doesn't find an optimal configuration faster at all, it has a slightly worse Validation Loss and doesn't seem to find a good one very early. This algorithm may not be the best for our case because there is not a very high number of Hyperparameters to optimize, however the execution time do not increases exponentially (unlike the classic Q-Learning), so for bigger problems with hundreds of hyperparameters to optimize it will become necessary.

Fourth Algorithm: Parallelization

In Reinforcement Learning, the "bottleneck" (the slow part) is usually the environment interaction. In our case, training the NanoGPT model takes 99% of the time while the Q-Learning or the DQN math takes 1%.

The new strategy, the "Batch Parallelism" is the following:

Instead of running Episode 1 → Train → Update → Episode 2, we will do this:

1. Collect Phase: The Agent quickly selects configurations for 4 separate episodes (e.g. Config A, B, C, D)

2. Parallel Phase: we send these 4 configs to 4 separate CPU cores. They train simultaneously.

3. Update phase: When the results come back, we update the Q-Network with all 4 results once. (We tested multiple configurations, but the Parallel DQN was the best parallel configuration).

This model finds a very good solution and explores a wide set of possibilities to even sometime bad solutions to avoid being stuck in a local minimum.

Best Loss: 2.5112. This algorithm has a better Validation Loss than any other Algorithm. However, it is less efficient, the execution takes 5 times more time than the other algorithms, which is A LOT more. Our attempt at parallelism failed because distributing AI training across these CPU cores (slower than GPU cores) proved far less efficient than utilizing the single dedicated GPU sequentially. The marginal performance gain of Parallel DQN (0.009 improvement) did not justify its 5x computational cost, especially given that the improvement falls within the margin of error (≈ 0.01). In the end, the most logical thing to do for our problem is to implement the first Algorithm: The simple Q-learning Algorithm.

Empirical Evaluation

As explained previously, we implemented a Genetic Algorithm as well as a Reinforcement Learning algorithm. We then compared the two, which were intended to find the best hyperparameters for training the NanoGPT on the TinyShakespeare dataset for predicting the next character.

Here are our results:

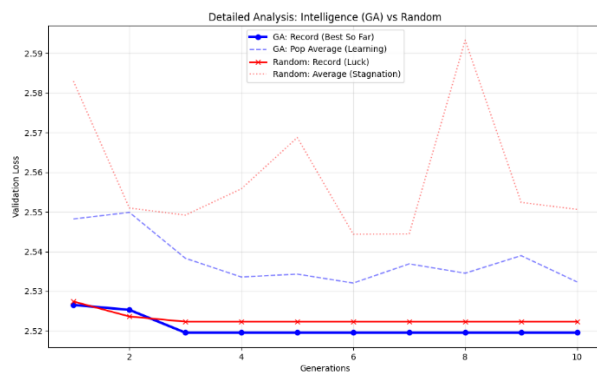


Figure A: GA – Best Loss = 2.5195
Finished in 2632.7s $\approx$ 44min

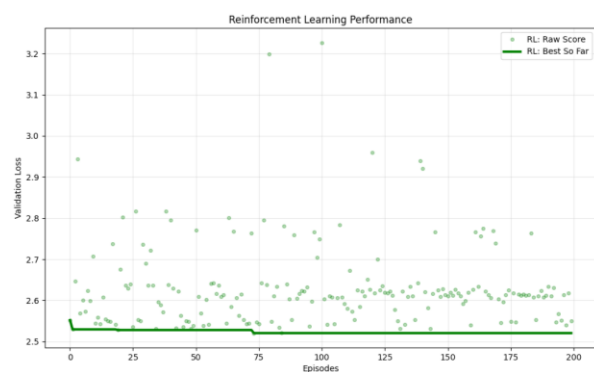


Figure B: RL (Q-learning) – Best Loss = 2.5206
Finished in 580.4s $\approx$ 10min

We chose Validation Loss as the primary metric for our problem. It guarantees that the hyperparameters we found create a model that actually learns the structure of the TinyShakespeare dataset, rather than just memorizing the specific sequence of characters in the input file.

These figures show that the GA and the RL have plateaued around a Validation Loss of 2.52. A limitation of our implementation is that a Validation Loss of 2.52 is not a great Loss for our model. However, we didn't really "train" our NanoGPT on the TinyShakespeare dataset, we "micro-trained" it. Basically, we essentially said: "Create a new brain, train it for a few minutes, tell me how smart it is, then kill it. Repeat 200 times". But each time the "new brain" created had different Hyperparameters.

The next step of the project could be to take our best Hyperparameters set and really train the NanoGPT with it, but this would take too long to run on our laptops.

The ceiling of 2.52 can be considered approximately the best possible Validation Loss for our problem. Moreover, this result of 2.52 has been found before half of the evaluations! It means that both GA and RL do not need 200 evaluations to find a good Hyperparameters set, they only need 100, this could highly reduce the execution time. Speaking of execution time, a big difference between the 2 implementations was noticed. While the GA took 44min, the RL took only 10min to execute. Which is a high ratio (4.5).

To understand how our approaches would perform on larger industrial problems (e.g., optimizing LLaMA-7B instead of NanoGPT), we analyzed their scalability regarding two factors: search space complexity and computational cost.

- **Impact of Search Space Resolution:** While Genetic Algorithms handle high-dimensionality naturally through mutation, Reinforcement Learning agents (specifically Q-Learning) suffer more from the "curse of dimensionality." As the number of discrete choices increases, the Q-table grows linearly, requiring significantly more episodes to converge. For larger models with hundreds of hyperparameters, a Deep Q-Network (DQN) becomes strictly necessary over tabular Q-Learning to approximate the value function efficiently.
- **Computational Scalability via Parallelism:** Both methods demonstrated high potential for horizontal scaling. Our implementation of Parallel DQN showed that RL sampling can be decoupled from model training. In a real-world scenario with a cluster of GPUs, the "Training" step (the bottleneck) can be distributed across hundreds of workers while a central RL agent or GA population manager updates the policy/genome. This suggests that while the cost of evaluation increases with model size, the wall-clock time can be kept manageable by adding more compute nodes, validating the approach for large-scale LLM deployment.

Conclusion

Our survey of the state-of-the-art literature reveals three distinct evolutionary trends in hyperparameter optimization (HPO) for Large Language Models:

- **From Static to Dynamic:** The field is moving away from static grid searches toward dynamic, lifecycle-aware methods. Techniques like Population Based Training (PBT)

and Reinforcement Learning schedules demonstrate that optimal hyperparameters are not fixed constants but trajectories that must adapt as the model matures.

- **Hybrid Efficiency:** Pure exploration (Random Search) is too slow, and pure exploitation (Bayesian Optimization) scales poorly with high dimensionality. The trend, exemplified by BOHB, is to hybridize these approaches—using "cheap" approximations (Hyperband) to filter the search space before applying expensive "smart" optimizers.
- **LLMs Optimizing LLMs:** The most cutting-edge trend involves using the semantic capabilities of LLMs to aid optimization. As seen in the Language-Aided Bayesian Optimization framework, translating numerical configurations into natural language allows optimization agents to leverage domain knowledge, significantly outperforming traditional black-box methods.

Through our implementation of Genetic Algorithms and Reinforcement Learning on NanoGPT, we demonstrated that "Micro-training" is a viable proxy for full-scale optimization.

- **Efficiency vs. Performance:** We found that both GA and RL could identify promising Hyperparameters configurations (Loss ≈ 2.52) in under 100 evaluations. This highlights that it is not necessary to fully train a model to determine if a hyperparameter set is promising or not.
- **Algorithm Behavior:** The Genetic Algorithm provided a robust, "staircase" convergence pattern fueled by elitism, making it a safe choice for stability. In contrast, Reinforcement Learning offered a faster execution time.

To bridge the gap between our academic experiments and industrial application, future work should focus on two areas:

1. **Transfer Learning for HPO:** Instead of training the RL agent from scratch for every new model, research should focus on developing a "Meta-Teacher." Similar to the Learning to Teach approach, a policy learned on a small model (like NanoGPT) could be transferred to initialize the optimization of larger models (like LLaMA 7B), saving thousands of GPU hours.
2. **Parameter-Efficient Fine-Tuning (PEFT):** Applying our GA and RL agents to optimize LoRA ranks and alphas. Since full fine-tuning is cost-prohibitive, automating the search for low-rank adapters would provide the highest ROI for democratizing LLM research.